

《计算机组成原理》实验报告

年级、专业、班级	大二计算机科学与技术 03、01、02	姓名	严浩睿、霍秋明、吴林兴
实验题目	实验一简单流水线与运算器实验		
实验时间	2026 年 4 月 12 日	实验地点	DS1421
实验成绩	优秀/良好/中等	实验性质	☐ 验证性 ☒ 设计性 ☐ 综合性
教师评价： ☒ 算法/实验过程正确； ☒ 源程序/实验内容提交； ☒ 程序结构/实验步骤合理； ☒ 实验结果正确； ☒ 语法、语义正确； ☒ 报告规范； 其他: 评价教师: 吴长泽			
实验目的 (1)理解流水线设计原理； (2)了解算术逻辑单元 ALU 的原理； (3)熟悉并运用 Verilog 语言设计 ALU； (4)熟悉并运用 Verilog 语言设计流水线全加器；			

报告完成时间: 2026 年 5 月 5 日

1 实验内容

1.1 ALU 设计实验

实验要求实现以下算术运算功能,其对应的控制码及功能如下:

F _{2:0}	功能	F _{2:0}	功能
000	A + B(Unsigned)	100	$\overline{A}$
001	A - B	101	SLT
010	A AND B	110	未使用
011	A OR B	111	未使用

表 1: 算数运算控制码及功能

实验要求:

1. 根据 ALU 原理图,使用 Verilog 语言定义 ALU 模块,其中输入输出端口参考实验原理,运算指令码长度为 [2:0]。
2. 仿真时 B 端口输入为 32h'01, A 端口输入参照 4.1 中表格
3. 实现 SLT 功能。
4. 验证表 1 中所有功能。
5. 给出 RTL 源程序(.v 文件)

1.2 流水线实验

本次实验为仿真实验,设计完成后仅需进行行为仿真。

实验要求:

1. 实现 4 级流水线 32bit 全加器,每级处理 8 位数据,需带有流水线暂停和刷新;
2. 模拟流水线暂停,仿真时控制 10 周期后暂停流水线 2 周期,流水线恢复流动;
3. 模拟流水线刷新,仿真时控制 15 周期时流水线刷新。

2 实验设计

2.1 ALU

2.1.1 功能描述

ALU 模块实现 32 位数据的算术和逻辑运算。模块接收两个 32 位操作数 A 和 B,以及 3 位控制码 F[2:0],输出 32 位运算结果 Result。支持 6 种运算:无符号加法、减法、按位与、按位或、按

位取反和小于置位(SLT)。

根据指导书,ALU 有两种实现方式:

- **assign** 实现: 使用连续赋值语句和三目运算符, 适合纯组合逻辑的简单运算
- **always** 实现: 使用 `always@(*)` 块和 `case` 语句, 结构更清晰, 便于扩展

本实验采用 `always` 实现方式, 扩展支持 SLT 运算。

2.1.2 接口定义

表 2: ALU 模块接口定义

信号名	方向	位宽	功能描述
A	Input	32-bit	操作数 A
B	Input	32-bit	操作数 B
F	Input	3-bit	运算控制码, 选择 ALU 执行的运算类型
Result	Output	32-bit	运算结果, 输出至七段数码管显示

2.1.3 逻辑控制

ALU 采用纯组合逻辑实现, 通过 `case` 语句根据控制码 `F[2:0]` 选择对应运算:

F[2:0]	功能	说明
000	A + B (Unsigned)	无符号加法
001	A - B	减法
010	A AND B	按位与
011	A OR B	按位或
100	$\overline{A}$	按位取反
101	SLT	有符号比较, A < B 时输出 1, 否则输出 0
110	未使用	输出 0
111	未使用	输出 0

表 3: ALU 运算控制码及功能

SLT 运算使用 Verilog 内置函数 `$signed()` 将操作数转换为有符号数后进行比较, 按照补码规则判断大小。

2.2 4 级流水线 32bit 全加器

2.2.1 功能描述

实现 4 级流水线 32 位全加器。将 32 位加法运算拆分为 4 个流水阶段，每级处理 8 位数据的加法及进位传递，采用移位寄存器方案对齐各级的部分和输出。支持流水线暂停和刷新控制，用于模拟流水线冲突和异常处理。

流水线各级功能划分：

- **第 1 级：**计算低 8 位 (bit[7:0]) 的加法，产生部分和 sum_s1 及进位 cout_s1，剩余高位数据存入 rem_a1/rem_b1
- **第 2 级：**计算次低 8 位 (bit[15:8]) 的加法，接收第 1 级进位
- **第 3 级：**计算次高 8 位 (bit[23:16]) 的加法，接收第 2 级进位
- **第 4 级：**计算最高 8 位 (bit[31:24]) 的加法，接收第 3 级进位，输出最终进位 cout

由于各级部分和在不同时钟周期产生，需通过移位寄存器将各级结果对齐至同一时刻输出：

- 第 1 级的部分和 sum_s1 需延迟 3 个周期 (经过 sum1_d2→sum1_d3→sum1_d4)
- 第 2 级的部分和 sum_s2 需延迟 2 个周期 (经过 sum2_d3→sum2_d4)
- 第 3 级的部分和 sum_s3 需延迟 1 个周期 (经过 sum3_d4)
- 第 4 级的部分和 sum_s4 直接作为最高 8 位输出

2.2.2 接口定义

表 4: 流水线全加器接口定义

信号名	方向	位宽	功能描述
clk	Input	1-bit	时钟信号
cin_a	Input	32-bit	操作数 A
cin_b	Input	32-bit	操作数 B
cin	Input	1-bit	进位输入
stall	Input	1-bit	流水线暂停信号，高有效
flush	Input	1-bit	流水线刷新信号，高有效，优先级高于 stall
cout	Output	1-bit	进位输出
sum	Output	32-bit	加法结果和

2.2.3 逻辑控制

流水线控制机制：

1. **正常流动：**stall=0 且 flush=0 时，每个时钟周期数据向下一级推进。各级并行计算各自负责

的 8 位加法,进位在各级间传递

2. 暂停: $stall=1$ 时,所有流水线寄存器保持当前值不变,各级计算暂停
3. 刷新: $flush=1$ 时,所有流水线寄存器清零,包括各级部分和、进位、剩余数据以及移位寄存器。 $stall$ 和 $flush$ 同时有效时, $flush$ 优先

仿真控制时序:

- 第 10 个时钟周期后, $stall$ 信号有效 2 个周期,模拟第 2 级流水线暂停,观察流水线各级寄存器保持情况
- 第 15 个时钟周期时, $flush$ 信号有效 1 个周期,模拟第 3 级流水线刷新,观察流水线寄存器清零和重新流动

进位传递与结果拼接:

每级加法使用 {进位, 部分和} 的拼接格式: $\{cout_sN, sum_sN\} \leq \{1'b0, a_part\} + \{1'b0, b_part\} + carry_in$ 。进位 $cout_s1$ 从第 1 级传递至第 2 级, $cout_s2$ 传递至第 3 级, $cout_s3$ 传递至第 4 级。最终进位 $cout$ 由第 4 级产生。由于各级部分和在流水线中处于不同阶段,通过移位寄存器对齐后拼接为 32 位最终结果: $sum[7:0]$ 来自延迟 3 拍的 sum_s1 , $sum[15:8]$ 来自延迟 2 拍的 sum_s2 , $sum[23:16]$ 来自延迟 1 拍的 sum_s3 , $sum[31:24]$ 直接来自 sum_s4 。

3 实验过程记录

3.1 ALU 模块设计与仿真

工作内容:

1. 根据实验要求,使用 Verilog 语言编写 32 位 ALU 模块,支持加法、减法、与、或、非、SLT 共 6 种运算
2. 采用 `always@(*)` 块配合 `case` 语句实现组合逻辑,SLT 运算使用 `$signed()` 进行有符号比较
3. 编写 ALU 测试激励,将 B 端口固定为 32'h01, A 端口依次输入不同的测试值
4. 在 Vivado 中进行行为仿真,验证 6 种运算功能的正确性
5. 整理仿真波形,记录验证结果

3.2 流水线全加器设计与仿真

工作内容:

1. 设计 4 级流水线 32 位全加器,将 32 位加法划分为 4 个流水阶段,每级处理 8 位
2. 采用移位寄存器方案对齐各级部分和输出:第 1 级结果延迟 3 拍、第 2 级延迟 2 拍、第 3 级延迟 1 拍,最后与第 4 级结果拼接为 32 位输出
3. 实现 $stall$ 和 $flush$ 控制信号。 $stall$ 有效时所有寄存器保持; $flush$ 有效时清空全部寄存器,且 $flush$ 优先级高于 $stall$

4. 编写测试激励,前 10 个周期进行多组加法运算($0x11110000+0x22220000$ 至 $0x05550000+0x06660000$),第 10 周期后 stall 2 个周期,第 15 周期 flush 1 个周期,之后验证 $0x64+0xC8=0x12C$
5. 在 Vivado 中进行行为仿真,整理仿真波形

3.3 问题与解决方案

3.4 问题 1:SLT 运算的符号位处理

问题描述: SLT 指令需要将有符号数进行比较,但 ALU 输入为 32 位无符号类型。直接比较会将所有输入视为无符号数,导致比较结果错误。

解决方案:使用 Verilog 内置函数 \$signed() 将操作数 A 和 B 转换为有符号数后再进行比较,确保 SLT 按照补码规则正确比较大小。当 $A < B$ 时输出 1,否则输出 0。

3.5 问题 2:流水线暂停时数据保持

问题描述:当流水线暂停信号 stall 有效时,需要保持所有流水线级寄存器的值不变。如果在 stall 期间仍有新数据到达输入端,可能导致数据丢失或覆盖。

解决方案:在所有流水线寄存器的 always 块中加入 stall 判断条件。当 stall 有效时,寄存器保持原值不更新;只有 stall 无效时才正常推进数据。

3.6 问题 3:flush 与 stall 的优先级

问题描述:当 flush 和 stall 同时有效时,需要确定哪个信号优先级更高。

解决方案:flush 信号用于处理分支预测失败等异常情况,需要立即清空流水线,因此 flush 优先级高于 stall。在 Verilog 代码中,优先判断 flush 信号,只有在 flush 无效时才判断 stall。

3.7 问题 4:流水线各级部分和的对齐

问题描述:4 级流水线中,各级的部分和在不同时钟周期产生。如果直接将各级结果拼接,会出现不同批次数据的部分和混在一起,导致输出错误。

解决方案:采用移位寄存器方案。将第 1 级部分和依次经过 3 级移位寄存器,第 2 级经过 2 级,第 3 级经过 1 级,使所有部分和在同一时钟周期到达输出端。同时,各阶段的剩余操作数也需逐级传递,确保每级处理正确的数据段。

4 实验结果及分析

4.1 ALU 验证实验结果

ALU 验证实验将 B 端口固定为 32'h01, A 端口依次输入不同的测试值, 验证 6 种运算功能的正确性。实验结果如表 5 所示。

操作	F[2:0]	Num1 (A)	Result
A + B (Unsigned)	000	32'h00000002	32'h00000003
A - B	001	32'h000000FF	32'h000000FE
A AND B	010	32'h000000FE	32'h00000000
A OR B	011	32'h000000AA	32'h000000AB
$\overline{A}$	100	32'h000000F0	32'hFFFFFF0F
SLT	101	32'h00000081	32'h00000000

表 5: ALU 验证结果表

结果分析:

1. **A+B (Unsigned):** $0x00000002 + 0x00000001 = 0x00000003$, 结果正确
2. **A - B:** $0x000000FF - 0x00000001 = 0x000000FE$, 结果正确
3. **A AND B:** $0x000000FE \& 0x00000001 = 0x00000000$, 结果正确
4. **A OR B:** $0x000000AA | 0x00000001 = 0x000000AB$, 结果正确
5. **NOT A:** $\sim 0x000000F0 = 0xFFFFF0F$, 结果正确
6. **SLT:** $\$signed(0x00000081) = 129, \$signed(0x00000001) = 1, 129 < 1$ 为假, 输出 0, 结果正确

4.2 流水线阻塞仿真图

仿真中前 10 个周期正常进行加法运算, 输入数据依次为 $0x11110000 + 0x22220000$ 、 $0x22220000 + 0x33330000$ 等。第 10 个时钟周期后, stall 信号拉高并持续 2 个周期。stall 有效期间, 流水线所有寄存器保持原值不变, 新的输入数据无法进入流水线。2 个周期后 stall 信号拉低, 流水线恢复正常流动。

4.3 流水线刷新仿真图

stall 结束后继续正常运算。第 15 个时钟周期时, flush 信号拉高并持续 1 个周期。flush 有效期间, 流水线所有寄存器被清零, 包括各级部分和、进位、剩余数据以及移位寄存器。flush 结束后, 流水线从零状态重新开始流动, 后续输入 $0x64 + 0xC8$ 正常运算出结果 $0x12C$ 。

A ALU 代码 (alu.v)

```
module alu (  
    input  [31:0] A,  
    input  [31:0] B,
```

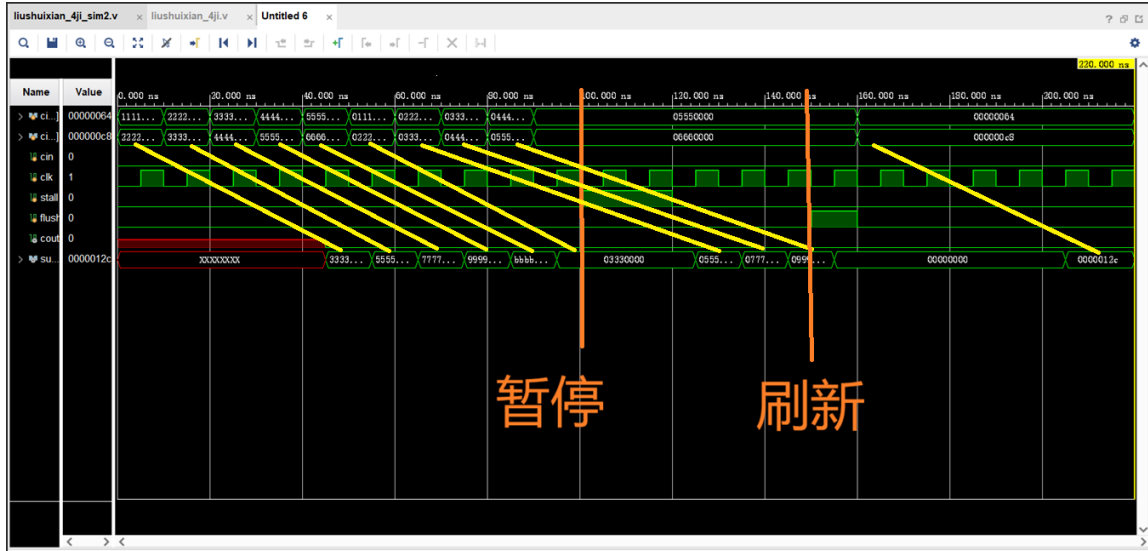


图 1: 流水线暂停仿真波形图

```

input  [2:0]  F,
output reg [31:0] Result
);

always @(*) begin
    case (F)
        3'b000: Result = A + B;
        3'b001: Result = A - B;
        3'b010: Result = A & B;
        3'b011: Result = A | B;
        3'b100: Result = ~A;
        3'b101: Result = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0;
        default: Result = 32'd0;
    endcase
end

endmodule

```

B 4 级流水线 32bit 全加器代码 (liushuixian_4ji.v)

```

module liushuixian_4ji (
    input [31:0] cin_a, cin_b,
    input cin, clk,
    input stall, flush,
    output reg cout,
    output reg [31:0] sum
);
    reg [7:0] sum_s1, sum_s2, sum_s3, sum_s4;
    reg [23:0] rem_a1, rem_b1;
    reg [15:0] rem_a2, rem_b2;

```



```

reg [7:0] rem_a3, rem_b3;
reg cout_s1, cout_s2, cout_s3, cout_s4;

reg [7:0] sum1_d2, sum1_d3, sum1_d4;
reg [7:0] sum2_d3, sum2_d4;
reg [7:0] sum3_d4;

always @(posedge clk) begin
    if (flush) begin
        sum_s1 <= 8'd0; cout_s1 <= 1'b0;
        rem_a1 <= 24'd0; rem_b1 <= 24'd0;

        sum_s2 <= 8'd0; cout_s2 <= 1'b0;
        rem_a2 <= 16'd0; rem_b2 <= 16'd0;

        sum_s3 <= 8'd0; cout_s3 <= 1'b0;
        rem_a3 <= 8'd0; rem_b3 <= 8'd0;

        sum_s4 <= 8'd0; cout_s4 <= 1'b0;

        sum1_d2 <= 8'd0; sum1_d3 <= 8'd0; sum1_d4 <= 8'd0;
        sum2_d3 <= 8'd0; sum2_d4 <= 8'd0;
        sum3_d4 <= 8'd0;

        sum <= 32'd0;
        cout <= 1'b0;
    end
    else begin
        if (!stall) begin
            // stage 1: bit [7:0]
            {cout_s1, sum_s1} <= {1'b0, cin_a[7:0]} + {1'b0, cin_b[7:0]} + cin;
            rem_a1 <= cin_a[31:8];
            rem_b1 <= cin_b[31:8];

            // stage 2: bit [15:8]
            {cout_s2, sum_s2} <= {1'b0, rem_a1[7:0]} + {1'b0, rem_b1[7:0]} +
                cout_s1;
            rem_a2 <= rem_a1[23:8];
            rem_b2 <= rem_b1[23:8];

            // stage 3: bit [23:16]
            {cout_s3, sum_s3} <= {1'b0, rem_a2[7:0]} + {1'b0, rem_b2[7:0]} +
                cout_s2;
            rem_a3 <= rem_a2[15:8];
            rem_b3 <= rem_b2[15:8];

            // stage 4: bit [31:24]
            {cout_s4, sum_s4} <= {1'b0, rem_a3} + {1'b0, rem_b3} + cout_s3;

```

```

        // shift register alignment
        sum1_d2 <= sum_s1;  sum1_d3 <= sum1_d2;  sum1_d4 <= sum1_d3;
        sum2_d3 <= sum_s2;  sum2_d4 <= sum2_d3;
        sum3_d4 <= sum_s3;

        // output assembly
        sum[7:0]    <= sum1_d4;
        sum[15:8]   <= sum2_d4;
        sum[23:16]  <= sum3_d4;
        sum[31:24]  <= sum_s4;
        cout <= cout_s4;

    end
end
end
endmodule

```

C 流水线全加器测试激励代码 (liushuixian_4ji_sim2.v)

```

module liushuixian_4ji_sim2();
    reg [31:0] cin_a, cin_b;
    reg cin, clk, stall, flush;
    wire cout;
    wire [31:0] sum;

    liushuixian_4ji uut (
        .cin_a(cin_a), .cin_b(cin_b), .cin(cin), .clk(clk),
        .stall(stall), .flush(flush), .cout(cout), .sum(sum)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0; stall = 0; flush = 0; cin = 0;

        // 1-5 cycles
        cin_a = 32'h11110000; cin_b = 32'h22220000; #10;
        cin_a = 32'h22220000; cin_b = 32'h33330000; #10;
        cin_a = 32'h33330000; cin_b = 32'h44440000; #10;
        cin_a = 32'h44440000; cin_b = 32'h55550000; #10;
        cin_a = 32'h55550000; cin_b = 32'h66660000; #10;

        // 6-10 cycles
        cin_a = 32'h01110000; cin_b = 32'h02220000; #10;
        cin_a = 32'h02220000; cin_b = 32'h03330000; #10;
        cin_a = 32'h03330000; cin_b = 32'h04440000; #10;
        cin_a = 32'h04440000; cin_b = 32'h05550000; #10;
        cin_a = 32'h05550000; cin_b = 32'h06660000; #10;
    end
endmodule

```

```
// stall at cycle 10 for 2 cycles
stall = 1;
#20;
stall = 0;
#30;

// flush at cycle 15
flush = 1;
#10;
flush = 0;

cin_a = 32'd100;
cin_b = 32'd200;

#60;
$finish;

end
endmodule
```